

Apuntes de Clase: Jetpack Compose

Estructura, composables y Material Design

Práctica de Clase

Jueves TN. Campus Montserrat

Materia: Desarrollo de Aplicaciones I – Lenguajes Visuales

Resumen

Este apunte desarrolla los conceptos fundamentales de Jetpack Compose para Android: el pasaje de interfaces imperativas a declarativas, la estructura de una aplicación Compose, la anatomía de una función `@Composable`, el manejo de estado, la recomposición, el patrón de elevación de estado y la importancia de Material Design como sistema visual y de interacción.

Índice

1. Introducción: ¿qué es Jetpack Compose?	3
2. De interfaces imperativas a interfaces declarativas	3
3. Estructura básica de una aplicación con Compose	4
4. ¿Qué es una función composable?	5
5. Elementos principales de un composable	6
5.1. Parámetros de entrada	6
5.2. Elementos visuales	7
5.3. Layout o estructura visual	7
5.4. Modifier	8
5.5. Estado	9
6. Recomposition: el corazón del modelo Compose	10
7. Buenas prácticas al escribir composables	11
8. Separación entre vista y lógica	12
9. State hoisting: elevar el estado	14

10. Material Design: qué es y por qué importa	15
11. Material 3 y MaterialTheme	16
12. Scaffold: estructura Material para una pantalla	17
13. Ejemplo completo: formulario simple con Compose y Material 3	18
14. Lectura técnica del ejemplo	20
15. Diferencia entre componente visual y lógica de negocio	21
16. Compose y arquitectura	22
17. Errores comunes al aprender Compose	22
18. Importancia académica de Compose	23

1 Introducción: ¿qué es Jetpack Compose?

Jetpack Compose es el toolkit moderno de Android para construir interfaces gráficas nativas usando Kotlin. A diferencia del modelo tradicional basado en archivos XML y clases View, Compose permite describir la interfaz directamente con funciones de Kotlin llamadas funciones composables. Google lo presenta como una herramienta para simplificar y acelerar el desarrollo de UI nativa en Android, usando menos código y APIs más intuitivas.

La idea central es que la interfaz se describe como una función del estado de la aplicación.

Dicho de forma sencilla:

```
UI = f(estado)
```

Esto significa que la pantalla que ve el usuario depende de los datos actuales del programa. Cuando esos datos cambian, Compose vuelve a ejecutar las partes necesarias de la interfaz para reflejar el nuevo estado. Ese proceso se llama recomposición.

En el modelo clásico de Android, muchas veces se modificaba manualmente una vista:

```
textView.text = "Hola"  
button.setOnClickListener { ... }
```

En Compose, en cambio, se describe cómo debería verse la interfaz para un estado dado:

```
Text(text = mensaje)
```

Si mensaje cambia, Compose actualiza la UI correspondiente.

Este cambio parece pequeño, pero conceptualmente es enorme. Compose no se basa tanto en “buscar una vista y modificarla”, sino en declarar qué interfaz corresponde al estado actual del programa.

2 De interfaces imperativas a interfaces declarativas

Tradicionalmente, Android usó un modelo imperativo. El programador definía una pantalla en XML y luego, desde Kotlin o Java, buscaba los elementos y los modificaba. Por ejemplo, se podía tener un TextView en XML y luego hacer:

```
val texto = findViewById<TextView>(R.id.texto)  
texto.text = "Resultado actualizado"
```

Ese enfoque funciona, pero a medida que la aplicación crece, puede volverse difícil mantener sincronizada la interfaz con el estado real del programa. El programador debe recordar qué vistas modificar, cuándo hacerlo y en qué orden.

Compose usa un modelo declarativo. En lugar de modificar manualmente los elementos visuales, se declara cómo debe verse la pantalla según los datos actuales.

Por ejemplo:

```
@Composable
fun PantallaSaludo(nombre: String) {
    Text(text = "Hola, $nombre")
}
```

Esta función no “busca” un `TextView`. Tampoco “setea” una propiedad de una vista ya existente. Simplemente describe que, para cierto valor de nombre, la pantalla debe mostrar un texto.

Según la documentación oficial, una función composable recibe datos y emite elementos de interfaz. Además, una función composable que representa UI normalmente no devuelve un valor, porque su finalidad es describir la pantalla, no construir manualmente objetos visuales para retornarlos.

3 Estructura básica de una aplicación con Compose

Una aplicación simple con Compose suele tener esta estructura:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }
}
```

```
        setContent {
            MiAplicacion()
        }
    }
}
```

La función `setContent { }` es el punto donde se define la interfaz usando Compose. Dentro de ese bloque se llaman funciones composables.

Un ejemplo más completo sería:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }
}
```

```
        setContent {  
            MaterialTheme {  
                PantallaPrincipal()  
            }  
        }  
    }  
}
```

```
@Composable  
fun PantallaPrincipal() {  
    Text(text = "Hola Jetpack Compose")  
}
```

Podemos distinguir tres niveles:

Primero, la Activity, que sigue siendo parte del ciclo de vida clásico de Android.

Segundo, el bloque setContent, que conecta la actividad con Compose.

Tercero, las funciones @Composable, que definen la interfaz.

En aplicaciones reales, además, se suele separar el código en varias capas: pantallas, componentes reutilizables, estado, ViewModel, navegación, tema visual y lógica de negocio.

Una estructura razonable para una app inicial podría ser:

```
app/  
  '-- src/main/java/com/ejemplo/app/  
    |-- MainActivity.kt  
    |-- ui/  
    |   |-- screens/  
    |   |   '-- PantallaPrincipal.kt  
    |   |-- components/  
    |   |   '-- TarjetaResultado.kt  
    |   '-- theme/  
    |       |-- Color.kt  
    |       |-- Theme.kt  
    |       '-- Type.kt  
  '-- data/  
    '-- ...
```

No es obligatorio organizarlo así desde el primer día, pero ayuda a que los estudiantes entiendan que Compose no significa “poner todo en un solo archivo”. Compose simplifica la UI, pero no elimina la necesidad de arquitectura.

4 ¿Qué es una función composable?

Una función composable es una función de Kotlin anotada con @Composable. Esa anotación le indica al compilador de Compose que la función participa en la construcción de la interfaz gráfica.

Ejemplo:

```
@Composable
fun Saludo(nombre: String) {
    Text(text = "Hola, $nombre")
}
```

Esta función tiene varios elementos importantes.

La anotación `@Composable` indica que la función puede emitir UI.

El parámetro `nombre: String` representa datos que entran a la función.

La llamada a `Text()` genera un elemento visual de texto.

La función no devuelve nada. No retorna un `View`, ni un `TextView`, ni un objeto visual explícito. Su tarea es describir qué debe mostrarse.

Un composable puede llamar a otros composables:

```
@Composable
fun PantallaPrincipal() {
    Column {
        Saludo(nombre = "Ana")
        Saludo(nombre = "Luis")
    }
}
```

Acá aparece una idea muy importante: una interfaz en `Compose` es una composición de funciones. Por eso el framework se llama `Compose`: se construyen pantallas combinando pequeñas piezas reutilizables.

5 Elementos principales de un composable

Un composable suele estar formado por varios elementos conceptuales.

5.1 Parámetros de entrada

Los parámetros permiten que el composable sea flexible y reutilizable.

```
@Composable
fun Mensaje(texto: String) {
    Text(text = texto)
}
```

Este composable no está atado a un mensaje fijo. Puede recibir distintos textos:

```
Mensaje("Bienvenido")  
Mensaje("Resultado correcto")  
Mensaje("Error de validación")
```

Esto es fundamental para diseñar buenos componentes. Un composable no debería depender innecesariamente de variables globales o datos ocultos. Lo ideal es que reciba lo que necesita por parámetro.

5.2 Elementos visuales

Compose incluye composables básicos como:

```
Text()  
Button()  
TextField()  
Image()  
Card()  
Column()  
Row()  
Box()
```

Por ejemplo:

```
@Composable  
fun FormularioSimple() {  
    Column {  
        Text(text = "Ingrese su nombre")  
        TextField(  
            value = "",  
            onChange = {}  
        )  
        Button(onClick = {}) {  
            Text(text = "Aceptar")  
        }  
    }  
}
```

Cada uno de estos elementos también es un composable. No son clases visuales tradicionales como `TextView` o `Button` en XML, sino funciones que describen UI.

5.3 Layout o estructura visual

Los layouts organizan los elementos en la pantalla.

Los más usados al inicio son:

```
Column  
Row  
Box
```

Column acomoda elementos verticalmente.

```
Column {  
    Text("Uno")  
    Text("Dos")  
    Text("Tres")  
}
```

Row acomoda elementos horizontalmente.

```
Row {  
    Text("Nombre:")  
    Text("Ana")  
}
```

Box permite superponer o posicionar elementos dentro de un contenedor.

```
Box {  
    Text("Contenido")  
}
```

La documentación oficial explica que Compose transforma el estado en elementos de UI a través de tres etapas: composición de elementos, layout de elementos y dibujo de elementos. También destaca que Compose ofrece layouts listos para usar y permite construir layouts personalizados cuando hace falta.

5.4 Modifier

Modifier es uno de los conceptos más importantes de Compose. Sirve para modificar
↪ apariencia, tamaño, posición, interacción y comportamiento de un componente.

Ejemplo:


```
Text(  
    text = "Hola",  
    modifier = Modifier  
        .padding(16.dp)  
        .fillMaxWidth()  
)
```

Con Modifier se pueden hacer cosas como:

```
Modifier.padding(16.dp)  
Modifier.fillMaxSize()  
Modifier.fillMaxWidth()  
Modifier.height(80.dp)  
Modifier.background(Color.LightGray)  
Modifier.clickable { }
```

Un patrón muy usado es pasar un modifier como parámetro del composable:

```
@Composable  
fun TarjetaUsuario(  
    nombre: String,  
    modifier: Modifier = Modifier  
) {  
    Card(modifier = modifier) {  
        Text(text = nombre)  
    }  
}
```

Esto permite que el componente sea reutilizable y adaptable. La función define qué muestra, pero quien la usa puede decidir cómo ubicarla o cuánto espacio darle.

Ejemplo:

```
TarjetaUsuario(  
    nombre = "Carlos",  
    modifier = Modifier.fillMaxWidth()  
)
```

Este patrón es muy importante en Compose porque evita componentes rígidos.

5.5 Estado

El estado representa datos que pueden cambiar durante la ejecución de la aplicación.

Por ejemplo, el texto ingresado por el usuario:

```
@Composable
fun FormularioNombre() {
    var nombre by remember { mutableStateOf("") }
}
```

```
TextField(
    value = nombre,
    onChange = { nuevoValor ->
        nombre = nuevoValor
    }
)
```

```
Text(text = "Hola, $nombre")
}
```

Acá aparecen dos elementos clave:

`remember` permite conservar un valor durante recomposiciones.

`mutableStateOf` crea un estado observable por Compose.

Cuando cambia nombre, Compose detecta el cambio y recompone las partes necesarias de la interfaz.

La documentación oficial remarca que Compose ayuda a ser explícito respecto de dónde y cómo se almacena el estado. También presenta el patrón de state hoisting, que consiste en mover el estado hacia el composible padre y dejar que el composible hijo reciba el valor y un evento de cambio.

6 Recomposition: el corazón del modelo Compose

La recomposición es el proceso por el cual Compose vuelve a ejecutar funciones composibles cuando cambian sus datos de entrada o su estado observado.

Ejemplo:

```
@Composable
fun Contador() {
    var contador by remember { mutableStateOf(0) }
}
```

```
Column {
    Text(text = "Valor: $contador")
}
```

```
Button(onClick = { contador++ }) {
    Text(text = "Incrementar")
}
```

```
}  
}  
}
```

Cuando el usuario presiona el botón, cambia el valor de contador. Compose detecta ese cambio y actualiza la parte de la UI que depende de ese valor.

No es necesario escribir:

```
textView.text = contador.toString()
```

Compose se encarga de actualizar la interfaz a partir del nuevo estado.

La documentación oficial explica que, cuando cambia el estado, las funciones composables pueden ser llamadas nuevamente con nuevos datos. Además, Compose intenta recomponer de manera inteligente solo las partes que cambiaron, evitando rehacer toda la interfaz innecesariamente.

Desde una perspectiva de ingeniería, esto obliga a pensar la UI de forma más funcional: la pantalla debe ser una consecuencia del estado, no una colección de objetos modificados manualmente.

7 Buenas prácticas al escribir composables

Una función composable debería ser, en general, rápida, predecible y sin efectos secundarios innecesarios.

Por ejemplo, esto no es una buena idea:

```
@Composable  
fun NumeroAleatorio() {  
    val numero = Random.nextInt(1, 100)  
    Text(text = "Número: $numero")  
}
```

¿Por qué? Porque Compose puede recomponer esa función muchas veces. Si cada recomposición genera un número nuevo, el resultado puede ser confuso e impredecible.

Es mejor separar el evento que modifica el estado:

```
@Composable  
fun NumeroAleatorio() {  
    var numero by remember { mutableStateOf(0) }  
}
```

```
Column {  
    Text(text = "Número: $numero")  
}
```

```
        Button(onClick = {
            numero = Random.nextInt(1, 100)
        }) {
            Text("Generar")
        }
    }
}
```

La regla pedagógica sería:

La UI describe.

Los eventos notifican.

La lógica modifica el estado.

El cambio de estado actualiza la UI.

Esto ayuda mucho a evitar que los estudiantes pongan demasiada lógica dentro de los elementos visuales.

8 Separación entre vista y lógica

En aplicaciones simples, es común escribir todo junto:

```
@Composable
fun PantallaSuma() {
    var numero1 by remember { mutableStateOf("") }
    var numero2 by remember { mutableStateOf("") }
    var resultado by remember { mutableStateOf("") }
}
```

```
Column {
    TextField(
        value = numero1,
        onChange = { numero1 = it }
    )
}
```

```
TextField(
    value = numero2,
    onChange = { numero2 = it }
)
```

```
Button(onClick = {
    val suma = numero1.toDouble() + numero2.toDouble()
    resultado = suma.toString()
})
```

```
    }) {  
        Text("Sumar")  
    }  
}
```

```
        Text(text = resultado)  
    }  
}
```

Para un primer ejemplo está bien, pero tiene problemas:

La vista contiene lógica de cálculo.

No hay validación.

Puede fallar si el usuario ingresa texto no numérico.

Es difícil de probar.

Una versión mejor separa la lógica:

```
fun sumar(a: String, b: String): String {  
    val numero1 = a.toDoubleOrNull()  
    val numero2 = b.toDoubleOrNull()  
  
    return if (numero1 != null && numero2 != null) {  
        (numero1 + numero2).toString()  
    } else {  
        "Ingrese números válidos"  
    }  
}
```

Y luego la UI la usa:

```
@Composable  
fun PantallaSuma() {  
    var numero1 by remember { mutableStateOf("") }  
    var numero2 by remember { mutableStateOf("") }  
    var resultado by remember { mutableStateOf("") }  
}
```

```
Column {  
    TextField(  
        value = numero1,  
        onChange = { numero1 = it },  
        label = { Text("Número 1") }  
    )  
}
```

```
    TextField(  
        value = numero2,  
        label = { Text("Número 2") }  
    )  
}
```

```
        onChange = { numero2 = it },  
        label = { Text("Número 2") }  
    )
```

```
    Button(onClick = {  
        resultado = sumar(numero1, numero2)  
    }) {  
        Text("Sumar")  
    }
```

```
        Text(text = resultado)  
    }  
}
```

Este enfoque es mucho más sano. La interfaz sigue siendo simple, pero la lógica ya no está enterrada dentro de la UI.

9 State hoisting: elevar el estado

Un concepto importante en Compose es el state hoisting, o elevación del estado. Consiste en sacar el estado de un composible hijo y pasarlo desde un composible padre.

Por ejemplo, un campo de texto podría escribirse así:

```
@Composable  
fun CampoNombre(  
    nombre: String,  
    onNombreChange: (String) -> Unit  
) {  
    TextField(  
        value = nombre,  
        onChange = onNombreChange,  
        label = { Text("Nombre") }  
    )  
}
```

Este composible no guarda su propio estado. Recibe el valor actual y una función para avisar cuando el usuario quiere cambiarlo.

Luego, el padre administra el estado:

```
@Composable  
fun PantallaFormulario() {  
    var nombre by remember { mutableStateOf("") }  
}
```

```
Column {  
    CampoNombre(  
        nombre = nombre,  
        onNombreChange = { nuevoNombre ->  
            nombre = nuevoNombre  
        }  
    )  
}
```

```
    Text(text = "Hola, $nombre")  
}  
}
```

Esto tiene varias ventajas:

El componente CampoNombre es más reutilizable.

La pantalla padre controla el estado general.

La lógica de la pantalla queda más clara.

Es más fácil testear y mantener.

La documentación oficial define este patrón como mover el estado al llamador del composable, reemplazando una variable interna por un par de parámetros: el valor actual y una función de cambio.

10 Material Design: qué es y por qué importa

Material Design es un sistema de diseño creado por Google para construir interfaces digitales consistentes, usables y visualmente coherentes. No es solo una colección de botones “lindos”. Es un lenguaje de diseño que incluye reglas sobre color, tipografía, espaciado, formas, movimiento, jerarquía visual e interacción.

En Compose, Material Design se implementa mediante componentes composables como:

```
Button  
Card  
Scaffold  
TopAppBar  
FloatingActionButton  
TextField  
NavigationBar  
Snackbar
```

La documentación oficial indica que Jetpack Compose ofrece una implementación de Material Design y que sus componentes se construyen como funciones composables.

La importancia de Material Design es técnica y pedagógica.

Desde el punto de vista del usuario, aporta consistencia. Una aplicación que usa correctamente Material Design se comporta de una manera familiar para quien ya usa Android.

Desde el punto de vista del desarrollador, reduce decisiones arbitrarias. En lugar de inventar desde cero cómo debe ser un botón, una tarjeta o una barra superior, se usan componentes ya diseñados, accesibles y coherentes.

Desde el punto de vista de ingeniería, Material Design favorece mantenibilidad. Una app construida con un sistema de diseño consistente es más fácil de modificar y escalar.

Desde el punto de vista de accesibilidad, ayuda a respetar criterios de contraste, tamaño, jerarquía visual e interacción, aunque esto no exime al desarrollador de validar correctamente la accesibilidad de su aplicación.

11 Material 3 y MaterialTheme

En Compose moderno se recomienda usar Material 3 para aplicaciones nuevas. La documentación oficial de Android señala que Material 3 es el conjunto más reciente de componentes Material para Compose, y que las apps nuevas deberían usar Material 3 en lugar de Material 2.

Una app Compose con Material 3 suele usar:

```
MaterialTheme {  
    PantallaPrincipal()  
}
```

MaterialTheme define el tema visual de la aplicación. En Material 3, el tema se
↪ organiza principalmente en tres subsistemas:

Color.

Tipografía.

Formas.

La documentación oficial explica que un tema M3 contiene esos tres subsistemas y que Compose los implementa mediante el composable MaterialTheme.

Ejemplo conceptual:

```
MaterialTheme(  
    colorScheme = colorScheme,  
    typography = typography,  
    shapes = shapes  
) {  
    App()  
}
```

Esto permite que los componentes Material usen una identidad visual coherente.

Por ejemplo:

```
Button(onClick = {}) {  
    Text("Aceptar")  
}
```

Ese botón no necesita definir manualmente todos sus colores, bordes, tamaños o estilos. Toma gran parte de su apariencia desde el tema Material activo.

12 Scaffold: estructura Material para una pantalla

Scaffold es un composable muy usado para estructurar pantallas Material. Permite

- ↪ organizar elementos comunes como barra superior, contenido principal, botón
- ↪ flotante, snackbar y barra inferior.

Ejemplo:

```
@OptIn(ExperimentalMaterial3Api::class)  
@Composable  
fun PantallaConScaffold() {  
    Scaffold(  
        topBar = {  
            TopAppBar(  
                title = { Text("Mi aplicación") }  
            )  
        },  
        floatingActionButton = {  
            FloatingActionButton(onClick = {}) {  
                Text("+")  
            }  
        }  
    ) { paddingValues ->  
        Column(  
            modifier = Modifier  
                .padding(paddingValues)  
                .padding(16.dp)  
        ) {  
            Text("Contenido principal")  
        }  
    }  
}
```

Scaffold ayuda a respetar una estructura visual común de Android. Además, maneja

- ↪ correctamente los espacios ocupados por barras superiores o inferiores mediante
- ↪ paddingValues.

Para estudiantes, este ejemplo es muy útil porque muestra que una pantalla no es solo una suma de textos y botones. Una pantalla real tiene estructura: encabezado, contenido, acciones principales y navegación.

13 Ejemplo completo: formulario simple con Compose y Material 3

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)
```

```
        setContent {  
            MaterialTheme {  
                PantallaPrincipal()  
            }  
        }  
    }  
}
```

```
@Composable  
fun PantallaPrincipal() {  
    var nombre by remember { mutableStateOf("") }  
    var mensaje by remember { mutableStateOf("Ingrese su nombre") }
```

```
    Scaffold(  
        topBar = {  
            TopAppBar(  
                title = { Text("Ejemplo Compose") }  
            )  
        }  
    ) { paddingValues ->
```

```
        Column(  
            modifier = Modifier  
                .padding(paddingValues)  
                .padding(16.dp)  
                .fillMaxSize(),  
            verticalArrangement = Arrangement.spacedBy(12.dp)  
        ) {  
            Text(  
                text = "Formulario de saludo",  
                style = MaterialTheme.typography.headlineSmall  
            )
```

```
TextField(  
    value = nombre,  
    onChange = { nombre = it },  
    label = { Text("Nombre") },  
    modifier = Modifier.fillMaxWidth()  
)
```

```
Button(  
    onClick = {  
        mensaje = if (nombre.isNotBlank()) {  
            "Hola, $nombre"  
        } else {  
            "Debe ingresar un nombre"  
        }  
    }) {  
    Text("Saludar")  
}
```

```
Card(  
    modifier = Modifier.fillMaxWidth()  
) {  
    Text(  
        text = mensaje,  
        modifier = Modifier.padding(16.dp)  
    )  
}
```

Este ejemplo integra varios conceptos:

MainActivity sigue siendo el punto de entrada de Android.

setContent activa Compose.

MaterialTheme aplica el sistema visual.

Scaffold organiza la pantalla.

Column acomoda los elementos verticalmente.

```
TextField captura entrada del usuario.
```

```
Button dispara un evento.
```

remember y mutableStateOf guardan estado local.

```
Card presenta información agrupada.
```

```
Modifier controla tamaño, padding y distribución.
```

14 Lectura técnica del ejemplo

La variable:

```
var nombre by remember { mutableStateOf("") }
```

representa el estado del campo de texto. Cada vez que el usuario escribe, se ejecuta:

```
onValueChange = { nombre = it }
```

El it representa el nuevo texto ingresado. Al asignarlo a nombre, cambia el estado. Como TextField depende de nombre, Compose puede recomponer esa parte de la UI.

La variable:

```
var mensaje by remember { mutableStateOf("Ingrese su nombre") }
```

guarda el mensaje que se muestra en la tarjeta. Cuando el usuario presiona el botón, se ejecuta:

```
mensaje = if (nombre.isNotBlank()) {  
    "Hola, $nombre"  
} else {  
    "Debe ingresar un nombre"  
}
```

Ese cambio actualiza automáticamente:

```
Text(text = mensaje)
```

La idea fuerte es esta: no se modifica manualmente la tarjeta ni el texto. Se modifica el estado, y Compose actualiza la UI.

15 Diferencia entre componente visual y lógica de negocio

Un error frecuente en estudiantes es creer que Compose es “solo dibujar pantallas”. En realidad, Compose obliga a pensar cuidadosamente qué pertenece a la interfaz y qué pertenece a la lógica.

Por ejemplo, esto está bien para un ejercicio inicial:

```
Button(onClick = {  
    mensaje = "Hola, $nombre"  
}) {  
    Text("Saludar")  
}
```

Pero en aplicaciones más serias, conviene que la lógica esté separada:

```
fun generarSaludo(nombre: String): String {  
    return if (nombre.isNotBlank()) {  
        "Hola, $nombre"  
    } else {  
        "Debe ingresar un nombre"  
    }  
}
```

Y luego:

```
Button(onClick = {  
    mensaje = generarSaludo(nombre)  
}) {  
    Text("Saludar")  
}
```

Esta separación permite probar la lógica sin depender de la UI.

Desde una perspectiva de ingeniería, esto es central. La interfaz debe ser una capa de presentación, no el lugar donde se mezclan validaciones, reglas de negocio, cálculos, persistencia y navegación sin control.

16 Compose y arquitectura

En aplicaciones más avanzadas, Compose suele integrarse con ViewModel.

Una estructura típica sería:

```
UI Composable
  ↓ eventos
ViewModel
  ↓ estado
UI Composable
```

La UI muestra el estado.

El usuario genera eventos.

El ViewModel procesa esos eventos.

El estado cambia.

Compose recompone la UI.

Conceptualmente:

```
@Composable
fun Pantalla(viewModel: MiViewModel) {
    val estado = viewModel.estado

    PantallaContenido(
        estado = estado,
        onEvento = { evento ->
            viewModel.procesar(evento)
        }
    )
}
```

Esta arquitectura permite separar responsabilidades:

El composable se ocupa de mostrar.

El ViewModel se ocupa de manejar estado y lógica de presentación.

El modelo o capa de dominio se ocupa de reglas más profundas.

La capa de datos se ocupa de persistencia, red o fuentes externas.

Compose no elimina la arquitectura. Al contrario: cuando la aplicación crece, una mala separación se nota muy rápido.

17 Errores comunes al aprender Compose

Un primer error común es poner demasiada lógica dentro del onClick.

```
Button(onClick = {  
    // muchas validaciones  
    // cálculos  
    // acceso a base de datos  
    // navegación  
    // modificación de muchas variables  
}) {  
    Text("Procesar")  
}
```

El `onClick` debería disparar una acción, no convertirse en el centro de la aplicación.

Otro error común es crear composables demasiado grandes. Si una función composable tiene cientos de líneas, probablemente debería dividirse.

También es común no usar `Modifier` como parámetro en componentes reutilizables. Eso hace que los componentes sean difíciles de adaptar.

Otro problema frecuente es no entender la recomposición. Un composable puede ejecutarse muchas veces, por lo que no conviene poner operaciones costosas o efectos secundarios directamente en el cuerpo de la función.

La documentación oficial advierte que las funciones composables deben ser rápidas, idempotentes y libres de efectos secundarios innecesarios, porque pueden recomponerse con frecuencia y `Compose` puede omitir recomposiciones cuando los parámetros no cambian.

18 Importancia académica de Compose

Desde el punto de vista formativo, `Compose` no es solamente una tecnología nueva de Android. Es una buena oportunidad para enseñar conceptos importantes de ingeniería de software:

Programación declarativa.

Separación de responsabilidades.

Estado y eventos.

Reactividad.

Composición de componentes.

Reutilización.

Diseño de interfaces.

Arquitectura de presentación.

Mantenibilidad.

Pruebas.

A diferencia del viejo enfoque XML más Activity con mucho código imperativo, `Compose` hace más visible la relación entre estado, eventos e interfaz. Eso ayuda a que los estudiantes entiendan que una app no es solo “pantallas con botones”, sino un sistema donde la UI representa un estado y permite disparar acciones controladas.